

LDSS Scholarship Monitoring System

CODEX FUNCTION-BY-FUNCTION IMPLEMENTATION PLAN

Engineering baseline for building, testing, and accepting the system in small, dependency-safe increments. This document is intentionally prescriptive so Codex can implement one function at a time without redefining scholarship policy.

INSTRUCTION TO CODEX

- Do not build the entire application in one pass. Implement the numbered functions in order unless a later function is explicitly independent.
- For each function: inspect current code/schema -> propose exact files/migrations -> implement -> run automated tests -> report changed files and test results -> stop for acceptance before moving to the next function.
- Never weaken a database constraint, RLS policy, audit rule, state transition, or approval rule merely to make the UI work.
- Frontend is untrusted. Authorization and workflow guards must be enforced server-side and, where practical, at database/RLS level.
- Never overwrite historical academic, masterlist, requirement, payout, OVR, approval, or audit records. Use versioning, amendments, corrections, or new period records.
- No production service-role key in frontend code. No shared user accounts. No public scholar directory or public scholar pages.

1. Product Boundary and Locked Rules

- Internal LGU-Daet scholarship monitoring system only. There is no scholar/student login or public application portal.
- The old iskolarnгдаet.app data is not authoritative. The new system establishes its own verified official masterlist.
- All documentary requirements are physical hard copies. The system records receiving, verification, correction, waiver, references, and storage location; it does not upload scholar documents.
- Every scholar receives one permanent human-readable Scholar ID in format LDSS-YYYY-00001 plus an internal UUID primary key. The year is first-entry year and never changes.
- Course shifting and school transfer preserve the same Scholar ID and create academic history/change records.
- Payout is semester-based. Default targets are first week of September for 1st Semester and February for 2nd Semester, but dates are configuration data, never hard-coded.
- COR and COG/Grades are recurring payout requirements. Requirement definitions are versioned/effective-dated.
- Official masterlists and finalized historical OVR batches are immutable. Changes occur through controlled amendment/version workflows.
- Eligibility is calculated from authoritative records. Manual override is a separate approval workflow and never destroys the original calculation.
- Important records are never hard-deleted. Use archive, void, cancel, correction, amendment, or status transition with reason.

2. Roles and Approval Authority

Capability	Staff	Coordinator	System Administrator
Create/edit scholar records	Yes	Yes	No by default
Receive COR/COG	Yes	Yes	No
Verify COR/COG	Yes	Yes	No
Prepare masterlist	Yes	Yes	No
Approve/publish/lock masterlist	No	Yes	No
Request masterlist amendment	Yes	Yes	No
Approve masterlist amendment	No	Yes	No
Request status change	Yes	Yes	No
Approve terminal status change	No	Yes	No
Request eligibility override	Yes	Yes	No
Approve eligibility override	No	Yes	No
Prepare OVR	Yes	Yes	No
Finalize/close OVR	No	Yes	No
Manage users/roles/configuration	No	No	Yes
View audit	Limited	Yes	Yes

Export audit	No	No	Yes
--------------	----	----	-----

System Administrator is a technical role, not scholarship approval authority. Scholarship approvals remain with the Coordinator even if the administrator can maintain infrastructure.

3. Authoritative State Models

Domain	States / Workflow
Qualification	Applicant -> Exam Passed -> Qualified -> Selected; Not Selected is a terminal outcome for that AY.
Scholarship	Active, On Hold, Suspended, Graduated, Dropped, Withdrawn, Disqualified, Not Renewed. Terminal states require controlled correction to reverse.
Requirements	Not Submitted -> Submitted -> For Verification -> Verified; correction loop: For Verification -> For Correction -> Resubmitted -> For Verification. Also Rejected or Waived.
Masterlist	Draft -> For Verification -> Submitted for Approval -> Approved -> Published -> Locked. Locked is immutable; amendments create a new version.
Eligibility override	None -> Override Requested -> Under Review -> Approved/Rejected -> Active Override; Active Override may be Cancelled by Coordinator.
OVR	Draft -> For Review -> Finalized -> Submitted -> Processed -> Closed. Finalization freezes membership; amendment is separate.

4. Technical Architecture

- Frontend: React + TypeScript + Bootstrap 5 / SB Admin Pro visual system.
- Database: PostgreSQL, preferably Supabase-managed if used by the project.
- Backend/API: controlled server-side service layer. Use /api/v1. Do not expose unrestricted table mutation to the browser.
- Authentication: individual accounts, secure sessions, account disable instead of delete, MFA for privileged users when feasible.
- Authorization: RBAC permissions plus PostgreSQL RLS/default-deny. Browser role labels are never trusted.
- Concurrency: optimistic expected_version plus database transactions/row locking for transitions and finalization.
- Audit: append-only audit_logs written in the same transaction as the business mutation.
- Environment separation: local/development, staging/UAT, production. Production secrets and DB access restricted.
- Backups: encrypted, restricted, scheduled, with restore tests documented.

5. Core Data Model

Table	Purpose / Critical constraints
users / roles / user_roles	Individual identities and RBAC. Disabled users retained.
scholars	Permanent person record; UUID PK; unique permanent Scholar ID.
scholar_contacts	Contact/address fields separated for controlled access.
academic_years / semesters	Period configuration and locking.
scholarship_records	One scholar + AY scholarship record; qualification/scholarship state.
academic_records	Scholar + AY school/course/year-level history; do not overwrite past AY.
academic_changes	Old/new values, type, effective date, reason, requester, approver.
masterlist_versions	AY, semantic version, workflow state, approval/publish/lock metadata.
masterlist_entries	Version membership, award/order number; no duplicate scholar in version.
masterlist_amendments	Immutable request/approval trail for changes to official versions.
requirement_definitions	Versioned/effective-dated requirement templates.
requirement_instances	Unique scholar + semester/payout + requirement; receiving/verification trail.
payout_cycles	Configurable semester payout target and workflow/lock metadata.
payout_eligibility	Calculated result, reason snapshot, final result, calculation timestamp/version.
eligibility_overrides	Requested/approved override, justification, authority reference, original result.
ovr_batches / ovr_entries	Frozen payout batch and scholar membership.
ovr_amendments	Changes after OVR finalization/submission without rewriting original.
payment_records	Per scholar/payout payment processing/reconciliation.
schools / courses / barangays	Controlled reference data.
audit_logs	Append-only event stream; no update/delete API.
system_settings	Non-secret configurable operational settings.

6. Codex Build Sequence

Each function below is a development checkpoint. Codex should stop after the acceptance gate of the current function. Do not jump directly to dashboards or payout screens before the underlying authoritative records and workflows exist.

F00 - Repository and Environment Baseline

Goal: establish a safe project skeleton and repeatable local/staging configuration.

Dependency	Database / Data	API / Service	Permission
None	Project migrations folder, env template, seed strategy.	Health endpoint; environment config loader.	Technical setup only.

Business rules

- Create React + TypeScript app structure and backend/service boundary.
- Define environment variables without committing secrets.
- Add lint, typecheck, unit-test and build scripts.
- Add database migration convention and seed convention.
- Create staging/production configuration separation.

Minimum tests

- Fresh clone installs successfully.
- Typecheck, lint, tests and production build execute.
- No secret/service-role key appears in frontend bundle or repository.

Acceptance gate

Codex reports repository structure, commands, environment variables required, and all baseline checks pass.

F01 - Authentication and Individual Accounts

Goal: only authenticated, active individual accounts may enter the system.

Dependency	Database / Data	API / Service	Permission
F00	users, auth provider mapping, disabled_at/active status.	login/session/logout/me endpoints or equivalent secure auth integration.	All roles; no anonymous access.

Business rules

- No shared accounts.
- Disabled users cannot start or continue authenticated sessions.
- Do not delete former staff identity because historical audit must resolve actor.
- Secure password reset/session handling; rate limiting where supported.

Minimum tests

- Anonymous request denied.
- Active account login succeeds.
- Disabled account denied.
- Expired/invalid session returns AUTH_REQUIRED or SESSION_EXPIRED.

Acceptance gate

All application routes are protected and actor identity is server-derived.

F02 - RBAC Permission Engine

Goal: enforce Staff, Coordinator, and System Administrator authority on the server.

Dependency	Database / Data	API / Service	Permission
F01	roles, permissions, user_roles or equivalent normalized model.	permission middleware/service; /me/permissions.	Per permission matrix.

Business rules

- Never trust role/permission values sent by browser.
- System Administrator is not Coordinator by implication.

- Default deny when permission is absent.
- RLS policies must align with server authorization.

Minimum tests

- Staff cannot approve masterlist.
- Coordinator can approve masterlist.
- SysAdmin cannot finalize OVR by default.
- Direct unauthorized API call returns 403 PERMISSION_DENIED.

Acceptance gate

Permission tests prove UI hiding is not the security boundary.

F03 - Reference Data and Academic Period Configuration

Goal: manage academic years, semesters, barangays, schools, courses, and system settings.

Dependency	Database / Data	API / Service	Permission
F02	academic_years, semesters, barangays, schools, courses, system_settings.	CRUD with archive rather than destructive delete.	SysAdmin manages; Staff/Coordinator read.

Business rules

- Academic year/semester codes unique.
- Referenced schools/courses cannot be destructively deleted.
- Period lock prevents prohibited mutations.
- Configuration dates stored as data.

Minimum tests

- Duplicate period rejected.
- Archived reference remains readable in history.
- Staff cannot change system configuration.

Acceptance gate

Reference data is usable by subsequent scholar forms without hard-coded lists.

F04 - Scholar Registry and Permanent Scholar ID

Goal: create authoritative scholar records with permanent IDs.

Dependency	Database / Data	API / Service	Permission
F03	scholars, scholar_identifiers, scholar_contacts.	POST/GET/PATCH /scholars; search endpoint with minimal PII.	Staff/Coordinator create/edit; SysAdmin no default scholarship edit.

Business rules

- Internal UUID is PK. Human ID format LDSS-YYYY-00001.
- ID sequence is concurrency-safe and unique.
- First-entry year in Scholar ID never changes.
- Important identity changes are audited.
- Search results expose only fields needed operationally.

Minimum tests

- Two concurrent creates cannot receive same Scholar ID.
- Duplicate Scholar ID DB constraint works.
- Editing AY does not alter Scholar ID.
- Unauthorized direct mutation blocked.

Acceptance gate

A scholar can be created, found, edited safely, and has an immutable permanent ID.

F05 - Duplicate Detection and Safe Scholar Creation

Goal: warn about possible duplicate persons without auto-merging.

Dependency	Database / Data	API / Service	Permission
F04	duplicate candidate metadata/log if persisted.	duplicate-check service used by create/import.	Staff/Coordinator.

Business rules

- Check combinations of name, DOB, barangay, contact and other available identifiers.
- Name match alone never auto-merges.
- User must explicitly resolve warning or cancel according to permission.
- All duplicate resolution decisions audited if record is created despite warning.

Minimum tests

- Exact likely duplicate flagged.
- Similar names are warnings, not forced merge.
- No duplicate Scholar ID possible regardless of warning bypass.

Acceptance gate

Creation flow prevents accidental duplicate records while preserving human review.

F06 - Scholarship Record and Qualification Workflow

Goal: track annual qualification separately from permanent person identity.

Dependency	Database / Data	API / Service	Permission
F04	scholarship_records, qualification events/status history.	command endpoints for exam-passed, qualify, select, not-select.	Staff prepares/records; Coordinator approval where required.

Business rules

- One scholarship record per scholar per AY.
- Passed != Qualified != Selected != Active.
- State transitions use command endpoints, not generic status PATCH.
- Selected -> Active only when official masterlist activation condition is satisfied.

Minimum tests

- Invalid transition returns 409 INVALID_STATE_TRANSITION.
- Duplicate AY scholarship record rejected.
- Staff cannot self-approve Coordinator-only transition.

Acceptance gate

Annual scholarship state is auditable and impossible to skip through unauthorized generic updates.

F07 - Scholarship Status Change Workflow

Goal: safely handle Active, hold, suspension and terminal outcomes.

Dependency	Database / Data	API / Service	Permission
F06	status_changes plus scholarship_records current state.	request/approve status command endpoints.	Staff request; Coordinator approve terminal/controlled changes.

Business rules

- Active -> On Hold/Suspended/Graduated/Dropped/Withdrawn/Disqualified/Not Renewed.
- Terminal states cannot be reversed by normal dropdown.
- Reversal requires controlled correction with reason/reference.
- Effective date, reason code, actor and approval are mandatory.

Minimum tests

- Dropped scholar cannot be normally eligible for payout.
- Staff terminal request does not become final without approval.
- Terminal reversal generic PATCH is rejected.

Acceptance gate

Status history is preserved and terminal states are protected.

F08 - Academic Records per Academic Year

Goal: preserve school/course/year-level history by AY.

Dependency	Database / Data	API / Service	Permission
F04,F03	academic_records.	academic record CRUD scoped to scholar + AY.	Staff/Coordinator.

Business rules

- Do not overwrite a prior AY record when current AY changes.
- One authoritative academic record per scholar/AY unless explicitly modeled otherwise.
- School/course must reference controlled data.

Minimum tests

- Adding new AY preserves prior AY.
- Missing required academic data blocks dependent workflows.
- Duplicate scholar/AY record rejected.

Acceptance gate

Scholar profile can display complete academic history.

F09 - Course Shift and School Transfer Workflow

Goal: change academic placement without changing Scholar ID or destroying history.

Dependency	Database / Data	API / Service	Permission
F08	academic_changes.	request/approve academic change commands.	Staff records/requests; Coordinator approval if configured for official changes.

Business rules

- Types: course shift, school transfer, both, year-level correction.
- Store old/new values, effective date, reason, remarks, requester, approver, approval date.
- Permanent Scholar ID unchanged.
- If an official locked masterlist is affected, route through masterlist amendment rather than silently changing official snapshot.

Minimum tests

- Course shift keeps same Scholar ID.
- History shows before/after values.
- Locked official data is not silently rewritten.

Acceptance gate

Academic changes are traceable and synchronized with official-record amendment rules.

F10 - Masterlist Draft Generation

Goal: build candidate masterlists from authoritative scholar records.

Dependency	Database / Data	API / Service	Permission
F06,F08	masterlist_versions, masterlist_entries.	create masterlist; add/remove candidates; validation endpoint.	Staff/Coordinator prepare.

Business rules

- 769 or any historical count is not assumed official.
- Candidate inclusion requires valid Scholar ID, AY scholarship record, selection state and academic record.
- Award number is separate from Scholar ID.
- Duplicate scholar in same version prohibited.

Minimum tests

- Duplicate entry rejected.
- Missing academic record reported as structured validation error.
- Candidate count derives from entries, not manual dashboard value.

Acceptance gate

Staff can prepare a clean draft and receive precise validation errors.

F11 - Masterlist Verification, Approval, Publish and Lock

Goal: create the official immutable masterlist through controlled approval.

Dependency	Database / Data	API / Service	Permission
F10	masterlist_versions approval/publish/lock metadata.	submit-for-verification, submit-for-approval, approve, publish, lock commands.	Staff submits; Coordinator approves/publishes/locks.

Business rules

- Workflow: Draft -> For Verification -> Submitted for Approval -> Approved -> Published -> Locked.
- Coordinator return to Draft requires reason.
- Approval requires version unchanged and no unresolved validation errors.
- Locked version cannot be edited.

Minimum tests

- Staff approval attempt denied.
- Version conflict returns 409 VERSION_CONFLICT.
- Locked entry mutation returns 423 RECORD_LOCKED.

Acceptance gate

An official masterlist can be produced and its locked snapshot is immutable.

F12 - Masterlist Amendment and Versioning

Goal: correct official masterlist data without rewriting history.

Dependency	Database / Data	API / Service	Permission
F11	masterlist_amendments; new masterlist version.	create amendment; approve/reject; publish amended version.	Staff request; Coordinator approve.

Business rules

- Example versions: 1.0 -> 1.1.
- Record original version, scholar, field/change, old/new, reason, requester, approver, dates/reference.
- Original locked version remains unchanged.
- Approved amendment creates or contributes to a new official version.

Minimum tests

- Original version byte/data snapshot unchanged after amendment.
- Rejected amendment changes nothing official.
- Approval actor cannot be spoofed from browser.

Acceptance gate

Official history can reconstruct what was true in every published version.

F13 - Requirement Definition Versioning

Goal: configure recurring hard-copy requirements without code changes.

Dependency	Database / Data	API / Service	Permission
F03	requirement_definitions, effective dates/version.	admin requirement definition endpoints.	SysAdmin configures; Staff/Coordinator read.

Business rules

- Requirements are effective-dated/versioned.
- COR and COG/Grades can be configured per semester/payout.
- Historical requirement instances retain the definition/version that applied then.
- Archive rather than delete used definitions.

Minimum tests

- Changing future requirement does not mutate past instance meaning.
- Inactive requirement cannot be newly instantiated outside effective range.

Acceptance gate

Requirement policy can evolve without corrupting history.

F14 - Requirement Instance Generation

Goal: create scholar-period requirement obligations from active definitions.

Dependency	Database / Data	API / Service	Permission
F13,F06	requirement_instances with uniqueness constraint.	generate/list scholar requirements for AY/semester/payout.	Staff/Coordinator.

Business rules

- Unique scholar + period + requirement definition/version.
- Initial status Not Submitted.
- Only applicable definitions generate instances.
- Generation is idempotent.

Minimum tests

- Repeated generation does not duplicate rows.
- Wrong-period definition not generated.

Acceptance gate

Every scholar/payout has a deterministic requirement checklist.

F15 - Physical Requirement Receiving

Goal: record receipt of hard-copy documents.

Dependency	Database / Data	API / Service	Permission
F14	requirement_instances receipt fields.	submit/receive command.	Staff/Coordinator.

Business rules

- Capture received date, received by, optional physical reference, folder/box/storage location, remarks.
- No binary document upload required.
- Already Verified/Waived or locked payout cannot be casually resubmitted.

Minimum tests

- Receipt creates audit event.
- Duplicate receipt is handled idempotently or rejected clearly.
- Locked payout blocks prohibited receipt change.

Acceptance gate

Physical document custody is traceable from receiving onward.

F16 - Requirement Verification and Correction Loop

Goal: verify COR/COG and manage correction/resubmission safely.

Dependency	Database / Data	API / Service	Permission
F15	verification fields/status history.	send-for-verification, verify, return-for-correction, resubmit, reject.	Authorized Staff/Coordinator verifier.

Business rules

- Validate scholar identity, AY/semester, school/course, applicability and document validity.
- Verified records store verifier/date/result/remarks.
- Verified -> For Correction is controlled correction and preserves original verification event.
- For Correction -> Resubmitted -> For Verification.

Minimum tests

- Unauthorized verifier denied.
- Wrong period returns REQUIREMENT_WRONG_PERIOD.
- Verified record satisfies normal requirement rule.

Acceptance gate

Requirement state machine works end-to-end and retains every decision.

F17 - Requirement Waiver

Goal: allow exceptional satisfaction of a requirement with explicit authority.

Dependency	Database / Data	API / Service	Permission
F16	waiver fields/status history.	waive command.	Coordinator only.

Business rules

- Waiver reason and authority/reference required.
- Waived satisfies requirement for normal eligibility unless payout policy explicitly says otherwise.
- Waiver never masquerades as Verified.

Minimum tests

- Staff waiver denied.
- Missing authority reference rejected.

Acceptance gate

Waivers are visible, attributable, and distinct from verification.

F18 - Payout Cycle Configuration

Goal: create configurable semester payout cycles.

Dependency	Database / Data	API / Service	Permission
F03	payout_cycles, payout_requirements.	create/configure/open/lock payout cycle.	SysAdmin configures dates; scholarship roles operate according to permission.

Business rules

- Default business targets may be September/February, but target date is stored per cycle.
- Cycle links AY and semester and required requirement definitions.
- Closed/locked cycles reject prohibited mutations.

Minimum tests

- Changing one cycle date does not affect historical cycles.
- Hard-coded payout date tests absent.

Acceptance gate

Each semester payout is an explicit configurable business object.

F19 - Calculated Payout Eligibility Engine

Goal: derive payout eligibility from authoritative records with explainable reasons.

Dependency	Database / Data	API / Service	Permission
F07,F11,F16,F17,F18	payout_eligibility calculation snapshot.	calculate/recalculate eligibility service and read endpoint.	Staff/Coordinator read; calculation server-owned.

Business rules

- Normal eligibility requires valid scholarship status, official masterlist condition, and all required requirement instances Verified or Waived.
- Graduated/Dropped/Withdrawn/Disqualified/Not Renewed cannot be normally eligible.
- Store calculation result, reason codes, source versions/timestamp.
- Dashboard/report must consume same authoritative calculation.

Minimum tests

- COR verified + COG missing => Not Eligible with reason.
- Both satisfied + Active + valid official list => Eligible.
- Dropped => Not Eligible regardless of documents.

Acceptance gate

Eligibility is deterministic, explainable, and reusable by OVR/reporting.

F20 - Eligibility Override Request and Approval

Goal: handle exceptional eligibility decisions without changing the original calculation.

Dependency	Database / Data	API / Service	Permission
F19	eligibility_overrides.	request, submit-review, approve, reject, cancel.	Staff request; Coordinator approve.

Business rules

- Preserve original calculated result.
- Require requested result, reason code, justification and authority reference.
- If source eligibility data changes while pending, require re-evaluation; do not approve stale request.
- Closed payout blocks new override.
- UI must label final state such as Eligible - Override Approved.

Minimum tests

- Staff cannot approve own request.
- Data change triggers ELIGIBILITY_DATA_CHANGED.
- Rejected override leaves calculation intact.

Acceptance gate

Exceptional decisions are explicit, approved, and fully auditable.

F21 - OVR Draft Batch and Entries

Goal: prepare the payout roster from currently eligible scholars.

Dependency	Database / Data	API / Service	Permission
F19,F20,F18	ovr_batches, ovr_entries.	create batch, add/remove entry, validate batch.	Staff/Coordinator prepare.

Business rules

- Adding entry causes fresh server eligibility check.
- Block ineligible scholar, incomplete requirements, invalid status, duplicate entry, unresolved pending override.
- Draft membership may change; finalized membership may not.

Minimum tests

- Duplicate entry rejected.
- Ineligible entry returns OVR_ENTRY_INELIGIBLE.
- Eligible entry succeeds.

Acceptance gate

Staff can build an OVR draft without bypassing eligibility.

F22 - OVR Review and Atomic Finalization

Goal: freeze an OVR only after every entry passes a fresh authoritative validation.

Dependency	Database / Data	API / Service	Permission
F21	OVN finalization metadata and immutable snapshot.	submit-review, return-to-draft, finalize.	Staff submits; Coordinator finalizes.

Business rules

- Finalization recalculates eligibility for every entry inside the transaction.

- All required COR/COG must be Verified/Waived.
- No pending overrides, invalid statuses, masterlist conflicts, or stale data.
- Atomic: one failed entry rolls back the whole finalization.
- Finalized membership frozen.

Minimum tests

- One invalid entry causes entire finalize to fail with structured errors.
- No partial finalized batch exists after failure.
- Staff finalize denied.

Acceptance gate

Coordinator can finalize a consistent immutable OVR snapshot.

F23 - OVR Submission, Processing and Closure

Goal: track the finalized OVR through actual administrative processing.

Dependency	Database / Data	API / Service	Permission
F22	OVR submission/processing/closure metadata, payment_records.	submit, mark-processed, close.	Coordinator submit/close; authorized Staff process.

Business rules

- Capture processing reference/date.
- Closure requires reconciliation confirmation and required payment records.
- Closed batch immutable.
- Closed payout blocks later ordinary override/membership mutation.

Minimum tests

- Unreconciled batch cannot close.
- Missing payment data blocks close where required.
- Closed mutation returns OVR_ALREADY_CLOSED/RECORD_LOCKED.

Acceptance gate

OVR lifecycle can be reconciled from preparation through closure.

F24 - OVR Amendment

Goal: handle corrections after finalization/submission without rewriting original OVR.

Dependency	Database / Data	API / Service	Permission
F22	ovr_amendments.	request/approve amendment commands.	Staff request; Coordinator approve.

Business rules

- Original finalized/submitted batch remains historically intact.
- Store old/new membership/data, reason, requester, approver, reference and timestamps.
- Amendment must respect payout closure policy.

Minimum tests

- Original entries remain reconstructable.
- Unauthorized approval denied.

Acceptance gate

Post-finalization correction is controlled and historically transparent.

F25 - Payment Record and Reconciliation

Goal: record payment outcome per scholar/payout cycle.

Dependency	Database / Data	API / Service	Permission
F23	payment_records unique scholar + payout cycle.	record/update processing result before closure; no destructive historical delete.	Authorized Staff/Coordinator.

Business rules

- One payment record per scholar/payout cycle.
- Capture status, amount if applicable, reference/date, remarks, actor.
- Correction after closed period uses controlled correction/amendment.

Minimum tests

- Duplicate payment rejected.
- Reconciliation detects OVR entry without required payment outcome.

Acceptance gate

Payout completion can be proven per scholar and reconciled to OVR.

F26 - Scholar Profile Timeline

Goal: give staff one authoritative view of a scholar without allowing unsafe shortcuts.

Dependency	Database / Data	API / Service	Permission
F04-F25	Read model/views only.	GET scholar profile/timeline aggregations.	Permission-aware Staff/Coordinator; SysAdmin only as policy permits.

Business rules

- Show personal info, academic history, scholarship history, masterlist membership, requirements, payouts, academic changes, status history and audit timeline.
- Do not expose mutation buttons the actor cannot use.
- Historical versions remain clearly labeled.

Minimum tests

- Profile totals/status match source records.
- Restricted PII omitted for insufficient permission.

Acceptance gate

A staff member can understand a scholar history without cross-checking spreadsheets.

F27 - Excel Import Staging and Validation

Goal: safely migrate/enter bulk scholar data without blind database writes.

Dependency	Database / Data	API / Service	Permission
F04,F05,F08	import_jobs, staging rows/errors if persisted.	upload/parse staging service; preview/confirm transaction.	Permission-controlled Staff/Coordinator.

Business rules

- Flow: staging -> validation -> duplicate/error report -> preview -> explicit confirmation -> transaction.
- Detect duplicate Scholar IDs, possible duplicate persons, missing fields, malformed dates, invalid references.
- Defend against spreadsheet formula injection on generated/exported files.
- Failed confirmed import rolls back transaction.

Minimum tests

- Bad row does not silently enter production.
- Duplicate warning shown before confirmation.
- Rollback test leaves no partial confirmed import.

Acceptance gate

Bulk import is reviewable, deterministic and transaction-safe.

F28 - Excel Export and PII Controls

Goal: export only authorized data and audit the disclosure.

Dependency	Database / Data	API / Service	Permission
------------	-----------------	---------------	------------

F02	export audit metadata.	permission-controlled export endpoints.	Specific permission; audit export reserved/restricted according to policy.
-----	------------------------	---	--

Business rules

- Minimize exported PII to report purpose.
- Audit who exported what, when, scope/filter and file/report type.
- Escape formula-like cell values where appropriate.

Minimum tests

- Unauthorized export denied.
- Export action appears in audit log.

Acceptance gate

Exports are useful operationally without becoming an untracked PII leak.

F29 - Operational Reports

Goal: produce reports from the same authoritative definitions used by workflows.

Dependency	Database / Data	API / Service	Permission
F19,F23,F25	SQL views/functions/read models.	report endpoints.	Role/PII scoped.

Business rules

- Examples: current scholars, missing requirements, for verification, eligibility, OVR status, paid/unpaid, graduates/dropped/disqualified, masterlist versions.
- No report reimplements eligibility differently from F19.
- Counts are database-derived, not manually stored dashboard numbers.

Minimum tests

- Report count reconciles to source query/view.
- Eligibility report exactly matches OVR eligibility checks.

Acceptance gate

Reports are reproducible and consistent with transactional screens.

F30 - Dashboard

Goal: surface operational workload and status without creating a second source of truth.

Dependency	Database / Data	API / Service	Permission
F29	Read-only dashboard views/cache if safe.	dashboard endpoint.	Role-scoped.

Business rules

- Metrics trace to authoritative records/views.
- Do not hard-code 769 or any historical total.
- Cards should link to filtered operational lists.
- Show current AY/payout context explicitly.

Minimum tests

- Dashboard count equals report/source count.
- Changing source record updates metric through same definition.

Acceptance gate

Dashboard is an operational lens, not an independent calculation layer.

F31 - Audit Log and Event Viewer

Goal: provide append-only traceability for every material action.

Dependency	Database / Data	API / Service	Permission
------------	-----------------	---------------	------------

F01 onward; implement event writing incrementally from first mutation	audit_logs append-only.	read/filter endpoint; no update/delete endpoint.	Staff limited, Coordinator view, SysAdmin view/export.
---	-------------------------	--	--

Business rules

- Minimum event: event_type, entity_type/id, action, from/to state, actor id/role, occurred_at, AY/semester/payout IDs, reason, remarks, reference, request_id.
- Write audit in same DB transaction as mutation.
- No application API can update/delete audit events.
- Amendments/overrides include old/new/original calculation details.

Minimum tests

- Rollback business transaction also rolls back its audit event.
- Direct audit update/delete is denied to app role.

Acceptance gate

Material changes are attributable and tamper-resistant at application level.

F32 - Concurrency, Idempotency and Locking

Goal: prevent double-click, stale-screen and simultaneous-user corruption.

Dependency	Database / Data	API / Service	Permission
All mutation functions	version columns, idempotency records/keys where needed.	Every state-changing command supports Idempotency-Key and expected_version.	All mutation permissions.

Business rules

- 409 VERSION_CONFLICT on stale expected_version.
- Repeated same Idempotency-Key does not duplicate mutation.
- Use transactions/row locks for finalization and sequential IDs.

Minimum tests

- Concurrent Scholar ID generation unique.
- Double finalization does not duplicate effects.
- Stale approval rejected.

Acceptance gate

Critical race conditions are covered by automated integration tests.

F33 - Error Contract and Validation UX

Goal: make business failures precise enough for staff and Codex debugging.

Dependency	Database / Data	API / Service	Permission
F00-F32	No new core table.	standard error envelope.	All.

Business rules

- HTTP: 400 malformed, 401 unauthenticated, 403 unauthorized, 404 missing, 409 state/concurrency, 422 business validation, 423 locked, 500 unexpected.
- Use stable codes: AUTH_REQUIRED, PERMISSION_DENIED, INVALID_STATE_TRANSITION, RECORD_LOCKED, VERSION_CONFLICT, MASTERLIST_VALIDATION_FAILED, REQUIREMENT_NOT_SATISFIED, NOT_ELIGIBLE, ELIGIBILITY_DATA_CHANGED, OVR_FINALIZATION_FAILED, etc.
- Never leak stack traces/secrets to client.

Minimum tests

- Each critical business guard maps to stable code.
- UI can show field/row-specific validation from structured details.

Acceptance gate

Failures are actionable and consistent across modules.

F34 - Security Hardening and RLS Review

Goal: verify that bypassing the UI does not bypass policy.

Dependency	Database / Data	API / Service	Permission
F01-F33	RLS policies, grants, security-definer functions only when justified.	API security review.	Default deny.

Business rules

- RLS enabled for sensitive tables.
- Service-role key never frontend.
- Search and list endpoints minimize PII.
- Rate limiting/session protections/MFA where feasible.
- Production DB direct access restricted.
- Free-text remarks treated as potentially sensitive.

Minimum tests

- Attempt cross-role direct DB/API access.
- Anonymous select/mutation denied.
- Frontend bundle scanned for privileged secrets.

Acceptance gate

Security test proves server/database, not UI, is the enforcement boundary.

F35 - Backup, Restore and Disaster Recovery

Goal: ensure the official scholarship record can be recovered.

Dependency	Database / Data	API / Service	Permission
Stable database schema	Backup metadata/procedure documentation.	Operational tooling, not public API.	SysAdmin restricted.

Business rules

- Encrypted backups.
- Retention schedule defined.
- Restore test to non-production environment.
- Document RPO/RTO targets and recovery steps.

Minimum tests

- Successful restore produces consistent counts/constraints.
- Audit/masterlist/OVR history survives restore.

Acceptance gate

A documented restore test is completed before production go-live.

F36 - UAT and Production Go-Live

Goal: validate real workflows before production.

Dependency	Database / Data	API / Service	Permission
F00-F35	UAT seed/test data and release metadata.	End-to-end tests and deployment checklist.	Named UAT users per role.

Business rules

- Run critical scenarios with Staff, Coordinator and SysAdmin accounts.
- No production launch with failing critical workflow/security tests.
- Freeze migration version, backup before release, define rollback.

Minimum tests

- Passed but incomplete scholar remains not payout eligible.
- Course shift keeps Scholar ID.

- Dropped scholar blocked from payout.
- Duplicate import rejected/warned correctly.
- COR verified + grades missing blocked.
- Locked masterlist immutable.
- OVR atomic finalization verified.
- Override approval audited.
- Previous semester paid history unchanged by new semester.

Acceptance gate

Coordinator and technical owner sign off UAT checklist; production release has backup and rollback plan.

7. API Command Contract

All workflow state changes use explicit command endpoints. Generic CRUD may edit non-state fields only where permitted.
Recommended base path: /api/v1.

Area	Representative commands
Qualification	POST /scholars/{id}/qualification/exam-passed; /qualify; /select
Scholar status	POST /scholars/{id}/status/on-hold; /suspend; /graduated; /dropped; /withdrawn; /disqualified; /not-renewed
Requirements	POST /requirements/{id}/submit; /send-for-verification; /verify; /return-for-correction; /resubmit; /reject; /waive
Masterlist	POST /masterlists; /{id}/submit-for-verification; /submit-for-approval; /approve; /publish; /lock; /{id}/amendments
Overrides	POST /payout-eligibility/{id}/override-requests; /eligibility-overrides/{id}/approve reject cancel
OVR	POST /ovr-batches; /{id}/add-entry; /remove-entry; /submit-review; /return-to-draft; /finalize; /submit; /mark-processed; /close; /amendments

Every state-changing request should carry an Idempotency-Key and expected_version. Transition service pattern: authenticate -> confirm active account -> server permission -> lock/load target -> validate current state/context/required fields -> enforce guard conditions -> mutate -> write audit event -> commit -> return authoritative result.

8. Required Stable Error Codes

Category	Codes
Authentication	AUTH_REQUIRED, ACCOUNT_DISABLED, SESSION_EXPIRED
Authorization	PERMISSION_DENIED, ROLE_NOT_AUTHORIZED, APPROVAL_REQUIRED, VERIFIER_NOT_AUTHORIZED
State/locking	INVALID_STATE_TRANSITION, STATE_ALREADY_CHANGED, RECORD_LOCKED, VERSION_CONFLICT
Masterlist	MASTERLIST_VALIDATION_FAILED, DUPLICATE_MASTERLIST_ENTRY, MASTERLIST_NOT_APPROVED, MASTERLIST_ALREADY_LOCKED
Requirements	REQUIREMENT_NOT_APPLICABLE, REQUIREMENT_ALREADY_SUBMITTED, REQUIREMENT_ALREADY_VERIFIED, REQUIREMENT_NOT_SATISFIED, REQUIREMENT_WRONG_PERIOD, UNAUTHORIZED_VERIFIER
Eligibility	NOT_ELIGIBLE, OVERRIDE_ALREADY_EXISTS, OVERRIDE_NOT_JUSTIFIED, ELIGIBILITY_DATA_CHANGED, PAYOUT_CYCLE_CLOSED
OVR	OVR_ENTRY_INELIGIBLE, OVR_REQUIREMENT_INCOMPLETE, OVR_DUPLICATE_ENTRY, OVR_FINALIZATION_FAILED, OVR_ALREADY_FINALIZED, OVR_ALREADY_CLOSED, OVR_RECONCILIATION_FAILED
Integrity	DUPLICATE_SCHOLAR, DUPLICATE_SCHOLAR_ID, MISSING_ACADEMIC_RECORD, FOREIGN_KEY_CONFLICT, INVALID_REFERENCE

9. Target Navigation and Screens

- Dashboard
- Scholars: All, Current, New, Continuing, Graduated, Dropped, Disqualified
- Masterlist: Current, Verification, Versions, Amendments

- Academic: Records, Course Changes, School Transfers
- Requirements: Monitoring, For Verification, Missing, For Correction
- Payouts: Current, Eligibility, OVR, Processing, Paid
- Reports
- Administration: Users, Academic Years, Requirements, Schools, Courses, Settings
- Audit Log

Build screens only after their backing function is authoritative. A polished screen must never precede or substitute for the required database constraints, transition services, permissions, and tests.

10. Prompt Template for Each Codex Function

Use the following instruction pattern when handing Codex one function. Replace [FUNCTION] with F00, F01, etc.

- Implement only [FUNCTION] from the LDSS implementation plan. Do not start the next function.
- First inspect the existing repository, schema, migrations, auth and tests relevant to this function. Reuse sound existing code; do not duplicate services.
- Before coding, state the exact files/migrations you will create or modify and any assumptions. Do not change locked scholarship policy.
- Implement database constraints/RLS first where applicable, then service/API rules, then UI, then automated tests.
- All state changes must be server-authorized, transactional where required, audited, concurrency-safe, and return the standard error contract.
- Run typecheck, lint, unit/integration tests and production build relevant to the change. Fix regressions caused by your changes.
- At completion, report: changed files; migrations; endpoints; permissions; tests run/results; manual test steps; unresolved risks. Then STOP and wait for approval.

11. Global Definition of Done

- Business rule is enforced in backend/database, not only UI.
- Permission and negative authorization tests exist.
- Database constraints prevent duplicate/impossible data where feasible.
- State transition uses explicit command service and validates current state.
- Expected-version/idempotency behavior exists for critical mutation.
- Audit event is written atomically with material mutation.
- No historical official record is overwritten.
- UI reflects server authority and handles structured errors.
- Automated tests cover success, invalid state, unauthorized role, stale version, and key business blockers.
- No PII is unnecessarily exposed in search, logs, errors, or exports.
- Lint/typecheck/tests/build pass before the function is accepted.
- Codex documents exactly what changed and stops before the next function.

12. Recommended Execution Checklist

ID	Function	Status
F00	Repository and Environment Baseline	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F01	Authentication and Individual Accounts	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F02	RBAC Permission Engine	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F03	Reference Data and Academic Period Configuration	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F04	Scholar Registry and Permanent Scholar ID	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F05	Duplicate Detection and Safe Scholar Creation	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F06	Scholarship Record and Qualification Workflow	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F07	Scholarship Status Change Workflow	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F08	Academic Records per Academic Year	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted

F09	Course Shift and School Transfer Workflow	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F10	Masterlist Draft Generation	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F11	Masterlist Verification, Approval, Publish and Lock	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F12	Masterlist Amendment and Versioning	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F13	Requirement Definition Versioning	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F14	Requirement Instance Generation	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F15	Physical Requirement Receiving	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F16	Requirement Verification and Correction Loop	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F17	Requirement Waiver	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F18	Payout Cycle Configuration	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F19	Calculated Payout Eligibility Engine	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F20	Eligibility Override Request and Approval	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F21	OVR Draft Batch and Entries	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F22	OVR Review and Atomic Finalization	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F23	OVR Submission, Processing and Closure	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F24	OVR Amendment	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F25	Payment Record and Reconciliation	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F26	Scholar Profile Timeline	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F27	Excel Import Staging and Validation	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F28	Excel Export and PII Controls	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F29	Operational Reports	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F30	Dashboard	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F31	Audit Log and Event Viewer	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F32	Concurrency, Idempotency and Locking	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F33	Error Contract and Validation UX	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F34	Security Hardening and RLS Review	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F35	Backup, Restore and Disaster Recovery	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted
F36	UAT and Production Go-Live	☐ Not started ☐ Implemented ☐ Tested ☐ Accepted